A Controller-Agnostic Lyapunov Safety Filter with Moving Offset-Free Targets, MPC Fallback, and Full Debug Instrumentation

Internal Technical Note

March 26, 2026

Abstract

This note documents the current Lyapunov safety-filter architecture implemented for the polymer CSTR repository. The method wraps a generic candidate controller, recomputes an admissible offset-free equilibrium at each sample using a refined target selector, tests the candidate action against hard bounds and a one-step Lyapunov decrease condition, and solves a minimally invasive correction problem only when needed. If the correction problem fails, the default backup is a fresh offset-free MPC action computed from the same observer state, setpoint, warm start, and constraints used by the baseline MPC path. The implementation now supports two canonical upstream controllers: a TD3 policy and a baseline offset-free MPC controller. In parallel, a Python debug/export layer records full simulation traces, target-selector metrics, Lyapunov margins, fallback behavior, solver statuses, and reproducible plots. The note develops the mathematics behind the method, maps the equations to the repository modules, and closes with concrete next actions for experimental validation.

1 Motivation and Scope

The repository already contains three ingredients that naturally belong together:

1. an offset-free disturbance-augmented prediction model and observer,
2. a baseline offset-free MPC controller,
3. a TD3-based reinforcement-learning controller.

What was missing was a single *verification-and-repair* layer that could sit downstream of either controller and enforce a local stability certificate without rewriting the upstream controller itself. That is the role of the Lyapunov safety filter.

The implemented design is intentionally different from a standard "terminal-cost MPC proof" perspective. The online object is not a full-horizon MPC law with a terminal set. Instead, the online object is a *controller-agnostic safety filter* of the form

$$\text{candidate controller} \longrightarrow \text{target selector} \longrightarrow \text{Lyapunov filter} \longrightarrow \text{plant}. \tag{1}$$

This places the main design burden on three questions:

1. how to define the correct moving equilibrium around which local stability should be checked,
2. how to verify a candidate action analytically before it is applied,

3. how to compute the smallest acceptable correction when the candidate fails.

The current implementation answers these questions with:

1. a refined Rawlings-style target selector, following the offset-free MPC literature [1, 2, 3, 4],
2. a physical-state quadratic Lyapunov function around the moving equilibrium,
3. a one-step convex correction problem inspired by safety-filter and CLF/CBF formulations [5, 6, 7],
4. a baseline offset-free MPC fallback for numerical failure or infeasibility of the repair solve.

2 Disturbance-Augmented Model and Observer

The repository uses a discrete-time disturbance-augmented linear model,

$$z_{k+1} = A_{\text{aug}} z_k + B_{\text{aug}} u_k, \tag{2}$$

$$y_k = C_{\text{aug}} z_k, \tag{3}$$

with augmented state

$$z_k = \begin{bmatrix} x_k \\ d_k \end{bmatrix}, \tag{4}$$

where $x_k \in \mathbb{R}^{n_x}$ is the physical state, $d_k \in \mathbb{R}^{n_y}$ is the disturbance model state, $u_k \in \mathbb{R}^{n_u}$ is the manipulated input, and $y_k \in \mathbb{R}^{n_y}$ is the measured output. In the codebase, the augmented matrices are formed as

$$A_{\text{aug}} = \begin{bmatrix} A & B_d \\ 0 & I \end{bmatrix}, \qquad B_{\text{aug}} = \begin{bmatrix} B \\ 0 \end{bmatrix}, \qquad C_{\text{aug}} = \begin{bmatrix} C & C_d \end{bmatrix}, \tag{5}$$

with $C_d = I$ for the standard offset-free augmentation used in the repository [1, 2].

The observer propagates an estimate

$$\hat{z}_k = \begin{bmatrix} \hat{x}_k \\ \hat{d}_k \end{bmatrix}, \tag{6}$$

and the notebooks store this as `xhatdhat`. The observer update used in the rollouts is

$$\hat{z}_{k+1} = A_{\text{aug}} \hat{z}_k + B_{\text{aug}} u_k + L\big(y_k - \hat{y}_k\big), \qquad \hat{y}_k = C_{\text{aug}} \hat{z}_k, \tag{7}$$

where the gain L is designed by pole placement on the augmented model.

Two remarks are essential for the rest of the note.

1. All controller logic is expressed in the repository's scaled deviation coordinates, not raw plant units.
2. The disturbance estimate $\hat{d}_k$ is not treated as a terminal nuisance parameter. It directly shapes the steady-state target used by the filter at every sample.

3 Moving Offset-Free Equilibrium

3.1 Target-Selector Problem

At each sample, the filter asks the refined target selector for an admissible equilibrium package

$$(x_s(k), u_s(k), d_s(k), y_s(k)), \qquad d_s(k) = \hat{d}_k. \tag{8}$$

The steady-state relation for the physical subsystem is

$$(I - A)x_s(k) - Bu_s(k) - B_d\hat{d}_k = 0, \tag{9}$$

and the corresponding steady-state output is

$$y_s(k) = Cx_s(k) + C_d\hat{d}_k. \tag{10}$$

The selector uses a two-stage design consistent with offset-free MPC target-selection practice [1, 2, 3]:

1. **Stage 1: exact target solve.** If possible, find a steady state that satisfies (9) and exactly matches the requested controlled output.
2. **Stage 2: closest-admissible target solve.** If the exact target is infeasible, solve a convex fallback problem that minimizes weighted target error while preserving steady-state feasibility and actuator bounds.

This separation is important conceptually. The target selector is the correct place to absorb steady-state infeasibility. The safety filter receives the selected target package as a frozen local reference and does not redefine the equilibrium internally.

3.2 Filter-Ready Target Package

The canonical output of the selector now includes:

1. x_s, u_s, d_s, and y_s,
2. success/failure status,
3. solve stage (`exact` or `fallback`),
4. target error and slack metrics,
5. residuals for the steady-state equations,
6. warm-start data for the next target solve.

The warm start is advisory only. The target is always recomputed from the current disturbance estimate; no cached target is ever treated as authoritative across changing disturbances.

4 Equilibrium-Centered Error Coordinates

The first implemented Lyapunov backend uses the physical-state error rather than the full augmented error:

$$e_x(k) = \hat{x}_k - x_s(k), \tag{11}$$
$$e_u(k) = u_k - u_s(k), \tag{12}$$
$$e_y(k) = y_k - y_s(k). \tag{13}$$

This choice is deliberate. It keeps the Lyapunov function tied directly to the physical subsystem regulated by the controller and avoids conflating physical stabilization with integrator/disturbance-estimator motion.

For a candidate input u, the one-step predicted physical error with frozen disturbance estimate is

$$e_x^+(u) = Ae_x(k) + B\big(u - u_s(k)\big). \tag{14}$$

The corresponding one-step predicted output is

$$y_{k+1|k}(u) = C\big(x_s(k) + e_x^+(u)\big) + C_d d_s(k). \tag{15}$$

Equation (14) is exactly the local model used by the candidate verifier and correction problem.

5 Lyapunov Function Design

5.1 Quadratic Lyapunov Candidate

The online filter uses the quadratic Lyapunov candidate

$$V_k = V(e_x(k)) = e_x(k)^\top P_x e_x(k), \tag{16}$$

where $P_x \succ 0$ is constructed from the physical subsystem (A, B) by solving a discrete algebraic Riccati equation. The code forms

$$Q_x = C^\top Q_y C + \varepsilon_x I, \tag{17}$$
$$R_\ell = \mathrm{diag}(r_1, \ldots, r_{n_u}), \tag{18}$$

with $Q_y \succeq 0$ inherited from output tracking weights and $R_\ell \succ 0$ either supplied explicitly or synthesized from input headroom. Then P_x solves

$$P_x = A^\top P_x A - A^\top P_x B\big(R_\ell + B^\top P_x B\big)^{-1} B^\top P_x A + Q_x. \tag{19}$$

The associated local LQR feedback is

$$K_x = -\big(R_\ell + B^\top P_x B\big)^{-1} B^\top P_x A. \tag{20}$$

5.2 Interpretation

This design follows standard MPC/LQR local-stability reasoning [8, 9], but its online role is narrower. We do not claim that the one-step filter itself replaces a full receding-horizon optimal-control proof. Instead, the Riccati solution is used to build a local quadratic certificate that is computationally cheap, physically interpretable, and available at every sample.

6 Candidate Acceptance Test

Let u_{cand} denote the action proposed by the upstream controller, either RL or MPC. The filter first computes

$$V_{k+1|k}(u_{\text{cand}}) = e_x^+(u_{\text{cand}})^\top P_x e_x^+(u_{\text{cand}}). \tag{21}$$

The hard acceptance condition implemented in the repository is

$$V_{k+1|k}(u_{\text{cand}}) - V_k \le -e_x(k)^\top Q_V e_x(k) - e_u(k)^\top R_V e_u(k), \tag{22}$$

which is equivalently evaluated through the bound

$$V_{k+1|k}(u_{\text{cand}}) \le \rho V_k + \varepsilon_\ell, \qquad 0 < \rho < 1, \quad \varepsilon_\ell \ge 0, \tag{23}$$

depending on the configured parameterization of the decrease test. The implemented default uses the latter bound form because it cleanly absorbs small equilibrium motion between samples through ε_ℓ.

The Lyapunov condition is enforced together with hard actuator and move constraints,

$$u_{\min} \le u_{\text{cand}} \le u_{\max}, \tag{24}$$

$$\Delta u_{\min} \le u_{\text{cand}} - u_{k-1} \le \Delta u_{\max}. \tag{25}$$

Hence a candidate is accepted unchanged if and only if:

1. it satisfies the actuator bounds (24),
2. it satisfies the move bounds (25),
3. it satisfies the Lyapunov decrease condition (22) or (23).

This is the source of the filter's minimally invasive behavior: the candidate passes through untouched whenever it is already acceptable.

7 Optimization-Based Correction

7.1 One-Step Repair QCQP

If the candidate is rejected, the filter solves a one-step convex repair problem of the form

$$\begin{aligned}\min_{u,\, s_t,\, s_v} \quad & (u - u_{\text{cand}})^\top W_c (u - u_{\text{cand}}) + (u - u_{k-1})^\top W_m (u - u_{k-1}) \\ & + (u - u_s(k))^\top W_s (u - u_s(k)) + \big(y_{k+1|k}(u) - y_{\text{track}}(k)\big)^\top W_y \big(y_{k+1|k}(u) - y_{\text{track}}(k)\big) \\ & + \lambda_t s_t^2 + \lambda_v s_v^2,\end{aligned} \tag{26}$$

subject to

$$u_{\min} \le u \le u_{\max}, \tag{27}$$

$$\Delta u_{\min} \le u - u_{k-1} \le \Delta u_{\max}, \tag{28}$$

$$V_{k+1|k}(u) \le \rho V_k + \varepsilon_\ell + s_v, \tag{29}$$

$$\|u - u_{\text{cand}}\|_\infty \le \delta_{\text{tr}} + s_t, \tag{30}$$

$$s_t \ge 0, \qquad s_v \ge 0. \tag{31}$$

The terms play distinct roles:

1. W_c penalizes deviation from the upstream candidate,
2. W_m suppresses aggressive step-to-step actuator motion,
3. W_s pulls the solution toward the selected equilibrium input u_s,
4. W_y optionally biases the one-step prediction toward the selected tracking output y_{track},
5. s_t provides trust-region flexibility only when trust-region slack is explicitly enabled,
6. s_v is a Lyapunov slack that is disabled by default in the main closed-loop path.

7.2 Hard-Default Philosophy

The implemented online default is intentionally strict:

$$s_v \equiv 0, \qquad s_t \equiv 0 \tag{32}$$

in the nominal execution path. That is, the correction problem is allowed to move the input, but not to violate the Lyapunov condition merely to obtain feasibility. Lyapunov slack remains available only for controlled debugging or continuation studies.

7.3 Solver Policy

Because the repair problem contains a quadratic Lyapunov constraint, the canonical solver order is:

$$\texttt{CLARABEL} \rightarrow \texttt{SCS}. \tag{33}$$

An `OSQP` path is reserved only for debug modes in which the quadratic Lyapunov constraint is dropped or replaced by a linearized surrogate.

8 Offset-Free MPC Fallback

8.1 Why the Fallback Changed

The earlier fallback order based on "previous safe input" or "clipped steady-state input" was deterministic, but it was not aligned with the baseline control logic already trusted in the repository. The current implementation therefore changes the *primary* failure policy:

> If the repair QCQP fails, compute a fresh offset-free MPC action using the same observer state, setpoint, warm start, bounds, and constraints as the baseline MPC path.

This produces a backup action that is operationally meaningful, rather than simply conservative.

8.2 Fallback Solve

Let $\hat{z}_k$ be the current observer state and u_{k-1} the previous applied input in deviation coordinates. The baseline MPC candidate is obtained from

$$\min_{U} \quad \sum_{j=0}^{N_P-1} \left\| y_{k+j+1|k}(U) - y_k^{\text{sp}} \right\|_{Q_y}^2 + \sum_{j=0}^{N_C-1} \left\| \Delta u_{k+j|k}(U) \right\|_{R_u}^2, \tag{34}$$

with the same horizon lengths, bounds, and initial guess used by the baseline offset-free MPC notebook and rollout code. The helper introduced in the Lyapunov path computes precisely one action:

$$u_{\mathrm{mpc,fb}} = \Pi_1(U^\star), \tag{35}$$

where Π_1 extracts the first control move from the optimized input sequence.

8.3 Verified and Unverified MPC Fallback Modes

After the fallback MPC candidate is generated, the filter subjects it to the same post-check as any other candidate:

$$\text{bounds check} + \text{move check} + \text{Lyapunov check}. \tag{36}$$

The resulting modes are:

1. `fallback_mpc_verified`: the MPC fallback satisfies the same hard checks and is applied as a verified safe action,
2. `fallback_mpc_unverified`: continuity requires applying the MPC fallback even though the Lyapunov certificate was not recovered.

Previous-safe and steady-state fallbacks are still retained as secondary emergency modes, but they are no longer the main runtime policy.

9 Dual Upstream-Controller Architecture

9.1 RL-Upstream Path

In the RL-facing rollout, the TD3 policy receives the RL state encoding already used elsewhere in the repository and proposes

$$a_k \in [-1, 1]^{n_u}. \tag{37}$$

This is mapped to the admissible deviation-input box,

$$u_{\mathrm{cand}} = \mathrm{map_to_bounds}(a_k, u_{\min}, u_{\max}), \tag{38}$$

and then passed to the target selector and safety filter. The filter may:

1. accept the RL action directly,
2. return an optimized correction,
3. fall back to a verified MPC action,
4. in continuity mode, return an unverified MPC fallback.

9.2 MPC-Upstream Path

In the MPC-facing rollout, the upstream candidate is itself the baseline offset-free MPC action,

$$u_{\text{cand}} = u_{\text{mpc}}. \tag{39}$$

This may seem redundant, but it serves two purposes:

1. it enforces a common Lyapunov-verification layer across all candidate controllers,
2. it provides a clean comparison between "baseline MPC" and "MPC filtered through the same safety certificate used for RL".

The same safety filter, the same target selector, the same debug payload, and the same MPC fallback logic are used for both upstream modes.

10 Debug and Export Schema

The new Python debug layer records every control step as a structured dictionary and then exports three artifact levels:

1. a full replay bundle (`bundle.pkl` and `arrays.npz`),
2. a flat per-step table (`step_table.csv`, and `step_table.pkl` when pandas is available),
3. a compact summary (`summary.json` and `summary.csv`).

10.1 Stored Per-Step Fields

For each step, the exported structure includes:

1. upstream source (`rl` or `mpc`),
2. candidate, corrected, applied, and MPC fallback inputs,
3. target package and target-selector diagnostics,
4. V_k, $V_{k+1|k}(u_{\text{cand}})$, V_{bound}, and the Lyapunov margin,
5. solver statuses and objective values,
6. slack values, trust-region usage, and correction mode,
7. verified/unverified flags and fallback reason,
8. measured outputs, observer states, disturbances, and rewards.

10.2 Derived Arrays and Plots

The debug module also derives and stores arrays for:

$$\text{target_error_inf}(k), \quad \text{target_slack_inf}(k), \quad V_{\text{bound}}(k) - V_{k+1|k}(u_{\text{cand}}), \quad \mathbb{I}\{\text{verified}\}(k), \tag{40}$$

and generates reproducible plots for:

1. outputs versus setpoints,
2. applied inputs,
3. candidate versus applied deviation inputs,
4. Lyapunov values and margins,
5. target-selector metrics,
6. reward traces,
7. correction-mode counts,
8. solver-status counts.

This makes post-run analysis independent of notebook-only helper cells.

11 Repository Mapping

The current code path is split across the following modules:

Module	Responsibility
`Lyapunov/target_selector.py`	Refined moving-equilibrium computation, two-stage Rawlings-style solve, filter-ready target package, selector diagnostics, warm-start packaging.
`Lyapunov/lyapunov_core.py`	Physical-model extraction, Riccati-based Lyapunov design, error-coordinate construction, one-step prediction, analytic candidate checks, PSD helper routines.
`Lyapunov/safety_filter.py`	Hard candidate verification, QCQP correction, verified/unverified fallback handling, shared public filter API.
`Lyapunov/upstream_controllers.py`	Single-step offset-free MPC candidate helper used by both the MPC-upstream rollout and the fallback path.
`Simulation/run_rl_lyapunov.py`	RL-upstream closed-loop rollout, notebook-facing entrypoint with stable return tuple.
`Simulation/run_mpc_lyapunov.py`	MPC-upstream closed-loop rollout using the same target selector and safety filter.
`Lyapunov/safety_debug.py`	Full run-bundle construction, flat step-table export, summary export, and plot generation.

12 Design Choices and Limitations

Several design choices are deliberate and should not be mistaken for oversights.

12.1 Physical-State Lyapunov Backend

The current online certificate is built on e_x, not the full augmented error. This improves interpretability and reduces numerical coupling between physical stabilization and disturbance-integrator motion. The main limitation is that the certificate does not yet reason directly about the augmented observer state.

12.2 One-Step Repair, Not Backup MPC

The correction problem is one-step and local. It is not intended to replace the baseline MPC optimizer. This makes the filter fast and controller-agnostic, but it can be conservative in situations where multi-step recovery is needed.

12.3 Hard Decrease by Default

The default path keeps the Lyapunov inequality hard. This is scientifically cleaner for the first implementation, but may reduce repair feasibility near aggressive setpoint changes or under poor local models.

12.4 Environment Constraints

The repository still lacks a pinned environment file and the default Python installation in the workspace does not yet provide the full scientific stack. This restricts runtime validation until the dependencies are installed.

13 Next Actions

The immediate next actions are as follows.

1. Install the missing runtime dependencies and execute end-to-end smoke tests for both the RL-upstream and MPC-upstream notebooks.
2. Compare closed-loop trajectories for four regimes: RL only, RL plus filter, MPC only, and MPC plus filter.
3. Quantify how often each mode is used: direct acceptance, optimized correction, verified MPC fallback, and unverified MPC fallback.
4. Add predictive backup-filter experiments that use a short-horizon safe backup solve when the one-step QCQP is too conservative.
5. Implement and compare an augmented-state Lyapunov backend against the current physical-state design.
6. Add regression scripts that automatically rebuild the debug bundle and summary metrics for a standard scenario.

14 Conclusion

The Lyapunov path in the repository is now organized as a canonical safety-filter architecture rather than an RL-specific notebook trick or a standard-tracking-MPC variant. The implemented method recomputes a moving admissible equilibrium each step, verifies the candidate action analytically, repairs it only when necessary, and falls back to a fresh offset-free MPC action when the repair optimizer fails. Just as importantly, the new debug/export module makes the full behavior inspectable after the fact. This combination gives the repository a clearer scientific story and a more reproducible experimental workflow.

References

[1] G. Pannocchia and J. B. Rawlings. Disturbance models for offset-free model-predictive control. *AIChE Journal*, 49(2):426–437, 2003.

[2] G. Pannocchia and A. Bemporad. Combined design of disturbance model and observer for offset-free model predictive control. *IEEE Transactions on Automatic Control*, 52(6):1048–1053, 2007.

[3] D. Limon, I. Alvarado, T. Alamo, and E. F. Camacho. MPC for tracking piecewise constant references for constrained linear systems. *Automatica*, 44(9):2382–2387, 2008.

[4] U. Maeder and M. Morari. Offset-free reference tracking with model predictive control. *Automatica*, 2010.

[5] A. D. Ames, K. Galloway, K. Sreenath, and J. W. Grizzle. Rapidly exponentially stabilizing control Lyapunov functions and hybrid zero dynamics. *IEEE Transactions on Automatic Control*, 59(4):876–891, 2014.

[6] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada. Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control*, 62(8):3861–3876, 2017.

[7] K. P. Wabersich and M. N. Zeilinger. A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica*, 129:109597, 2021.

[8] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, 2000.

[9] J. B. Rawlings, D. Q. Mayne, and M. Diehl. *Model Predictive Control: Theory, Computation, and Design*. Nob Hill Publishing, 2nd edition, 2017.